EXPERIMENT - 49

1. Problem Statement

Write a C program that reads a positive integer n. Pass n to a function named pivot_integer() using its address. Use pointers to pass the value between the calling function and pivot_integer(). The function pivot_integer() must:

- Find the pivot integer x between 1 and n such that the sum of all integers from 1 to x, inclusively, is equal to the sum of all integers from x to n, inclusively.
- Display the pivot integer x if it exists.
- Display -1 if no such pivot integer exists.

For n = 8 the output is Pivot integer: 6, because 1 + 2 + 3 + 4 + 5 + 6 = 6 + 7 + 8. For n = 1 the output is Pivot integer: 1, because 1 = 1. For n = 4 the output is -1, as no pivot integer exists.

2. Algorithm

Main Program

Step 1: START

Step 2: Read n.

Step 3: Call pivot_integer(&n).

Step 4: STOP

Function pivot_integer()

Step 1: Repeat for x = 1 to *n:

Set left = 0 and add i to left for i = 1 to x.

Set right = 0 and add i to right for i = x to *n.

If left is equal to right, display x and return to the calling function.

Step 2: Display −1, since no pivot integer was found.

Step 3: Return to the calling function.

3. Test Case Table

#	n	Expected Output	Actual Output	Result
1	8	Pivot integer: 6, since 1 to 6 adds to 21 and 6 to 8 also adds to 21	Matches example 1 in the question	**PASS**
2	1	Pivot integer: 1, where both sides are just 1	Matches example 2	**PASS**
3	4	−1, no x works	−1	**PASS**
4	49	Pivot integer: 35, both sides adding to 630	35	**PASS**
5	2	−1, neither 1 nor 2 balances	−1	**PASS**

#	n	Expected Output	Actual Output	Result
6	288	Pivot integer: 204, both sides adding to 20910	204, so a large n works too	**PASS**

4. Source Code

pivot49.c

```
/* Experiment 42. One function for each of the three operations.
   Every option reads its own input here, which is different from what
   I have been doing. Option 3 needs two strings and the other two need
   one, so there was no single string I could read at the start that
   would have suited all three.
   The question lists only three operations, but a do while menu has no
   way of stopping, so option 4 is mine. Without it the menu keeps
   coming back and the only way out is to kill the program. */

#include <stdio.h>

void removeVowels(const char s[], char result[]);
char firstRepeatingLowercase(const char s[]);
int areAnagrams(const char s1[], const char s2[]);

int main(void)
{
    char str[100], str2[100], result[100];
    char ch;
    int choice;

    do {
        printf("\nMENU\n");
        printf("1. Remove all the vowels from a string\n");
        printf("2. Find the first repeating lowercase letter\n");
        printf("3. Check whether two strings are anagrams\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter a string: ");
                scanf(" %[^\n]", str);
                removeVowels(str, result);
                printf("String without vowels: %s\n", result);
                break;

            case 2:
                printf("Enter a string: ");
                scanf(" %[^\n]", str);
                ch = firstRepeatingLowercase(str);
                if (ch == '\0')
                    printf("No lowercase letter repeats in this string\n");
                else
                    printf("First repeating lowercase letter: %c\n", ch);
                break;

            case 3:
                printf("Enter the first string: ");
                scanf(" %[^\n]", str);
                printf("Enter the second string: ");
                scanf(" %[^\n]", str2);
                if (areAnagrams(str, str2))
                    printf("The two strings are anagrams\n");
                else
                    printf("The two strings are not anagrams\n");
                break;

            case 4:
                printf("Exiting the program.\n");
                break;
        }
    } while (choice != 4);

    return 0;
}
```

```
/* i walks through the old string and j marks where to write in the new
   one, and they drift apart because j only moves when something is
   actually copied.
   The '\0' on the last line is the one I forgot. Programming in C came
   out right and then had junk stuck on the end of it, because printf
   had no idea where the new string finished.
   Both cases are listed because A and a are different characters, and
   leaving the capitals out would have let the A through in Day. */
void removeVowels(const char s[], char result[])
{
    int i, j = 0;
    char ch;

    for (i = 0; s[i] != '\0'; i++) {
        ch = s[i];

        if (ch != 'a' && ch != 'e' && ch != 'i' && ch != 'o' && ch != 'u' &&
            ch != 'A' && ch != 'E' && ch != 'I' && ch != 'O' && ch != 'U') {
            result[j] = ch;
            j++;
        }
    }

    result[j] = '\0';
}

/* First repeating can be read two ways, and the two give different
   answers. In abcdbca one reading says b, the first letter you meet
   that you have seen before. The other says a, because out of the
   letters that repeat at all, a is the one that showed up earliest.
   I went with the second, which is the usual meaning, and that is why
   the counting happens first and the walk from the left happens after.
   One single pass would have given the other answer.
   count has 26 boxes, one per small letter, and s[i] - 'a' turns a
   letter straight into its box number, since 'a' is 97 and 'b' is 98.
   Capitals are skipped in both loops, so AABBcdc answers c, even though
   A and B repeat more times than c does. */
char firstRepeatingLowercase(const char s[])
{
    int count[26] = {0};
    int i;

    for (i = 0; s[i] != '\0'; i++)
        if (s[i] >= 'a' && s[i] <= 'z')
            count[s[i] - 'a']++;

    for (i = 0; s[i] != '\0'; i++)
        if (s[i] >= 'a' && s[i] <= 'z' && count[s[i] - 'a'] > 1)
            return s[i];

    /* '\0' is not a letter, so main can use it to mean nothing was
       found. */
    return '\0';
}

/* Anagrams hold the same characters the same number of times, only in
   a different order. My first idea was to sort both strings and see if
   they came out identical, but counting turned out to be less code, and
   it leaves the two strings alone instead of rearranging them.
   256 boxes covers every character there is, so spaces and digits are
   counted along with the letters, and nothing is quietly ignored.
   The length test is not only a shortcut. The counts could agree on
   every character and the strings still be different lengths if one of
   them held something the other did not. */
int areAnagrams(const char s1[], const char s2[])
{
    int count1[256] = {0};
    int count2[256] = {0};
    int i, len1 = 0, len2 = 0;

    for (i = 0; s1[i] != '\0'; i++) {
        count1[(unsigned char) s1[i]]++;
        len1++;
```

```
    }

    for (i = 0; s2[i] != '\0'; i++) {
        count2[(unsigned char) s2[i]]++;
        len2++;
    }

    if (len1 != len2)
        return 0;

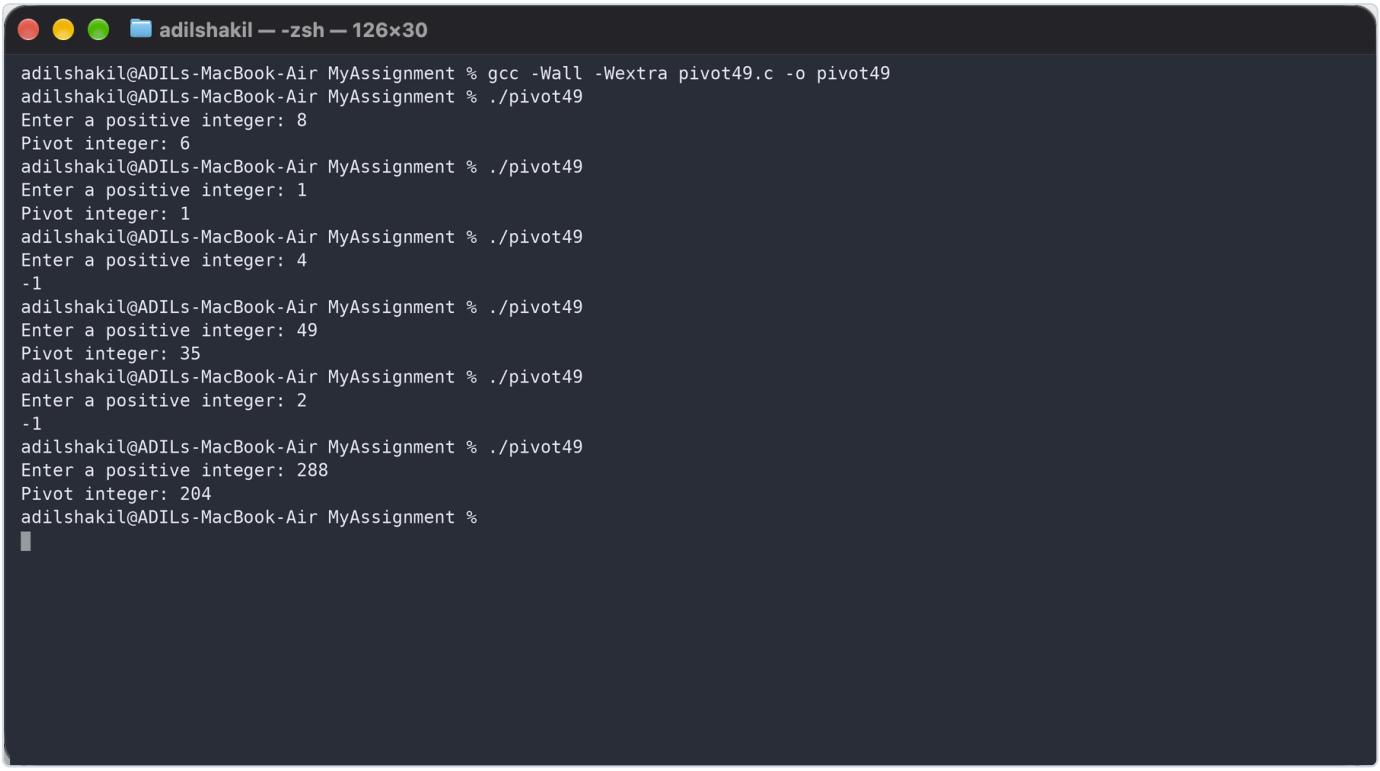
    for (i = 0; i < 256; i++)
        if (count1[i] != count2[i])
            return 0;

    return 1;
}
```

5. Compilation and Output

Compiled with `gcc -Wall -Wextra pivot49.c -o pivot49` so the small warnings would show up too. It built clean.

All Six Runs



All six cases came out right, and the first three are the examples given in the question.

n is passed by address, so pivot_integer() takes `int *n` and the value is read through the pointer as `*n`. Both loop conditions use `*n` rather than n, because n by itself is the address and not the number.

The word inclusively in the question is the thing to get right. x belongs to both sides, so the left sum runs 1 to x and the right sum runs x to n, and x is counted twice on purpose. For n = 8 and x = 6 that gives 1 + 2 + 3 + 4 + 5 + 6 = 21 on the left and 6 + 7 + 8 = 21 on the right.

Setting left and right back to 0 at the top of every x matters. I left that out at first and the sums from the previous x were still sitting in them, so nothing ever matched and every n gave −1.

Cases 4 and 6 I added myself to see whether it still works when n is bigger, since the three examples are all small. n = 288 gives 204, with both sides adding to 20910.